

2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В ходе прохождения учебной практики использовался язык программирования Python, среда разработки (IDE) PyCharm



Рисунок 2.1 – Логотип среды разработки PyCharm

2.1 Описание используемого языка программирования и IDE.

2.1.1 Описание языка программирования Python.

Python – это высокоуровневый, интерпретируемый язык программирования с простым и читаемым синтаксисом.

Преимущества Python:

- простота и читаемость синтаксиса;
- обширная экосистема готовых библиотек и фреймворков;
- многообразие областей применения;
- бесплатность и кроссплатформенность.

Недостатки Python:

- низкая скорость выполнения;
- проблемы с многопоточностью;
- слабые возможности статического анализа;
- меньше гарантий на этапе компиляции.

Для разработки игры «Палочки» выбор языка Python является оптимальным, так как он сочетает простоту и читаемость синтаксиса с

широкими возможностями для реализации игровой логики. Python позволяет быстро создавать прототипы, эффективно управлять игровым полем и реализовывать подсчёт очков и цепочки ходов. Наличие библиотек для графического интерфейса, работы с событиями и алгоритмов искусственного интеллекта ускоряет разработку и упрощает добавление интеллектуальных функций, таких как подсказки или компьютерный соперник. Кроме того, Python кроссплатформенен, что обеспечивает работоспособность игры на различных операционных системах, и предоставляет удобные возможности для образовательного и исследовательского применения проекта.

2.1.2 Описание используемой IDE.

PyCharm – это интегрированная среда разработки для языка программирования Python, разработанная компанией JetBrains. Данная среда широко используется как в учебной, так и в профессиональной разработке программного обеспечения. PyCharm предоставляет полный набор инструментов для создания, редактирования, запуска, отладки и сопровождения программных проектов на Python.

IDE предоставляет интеллектуальное автодополнение, подсказки и навигацию по коду, что помогает быстро находить ошибки и улучшать структуру кода. Встроенный отладчик и поддержка модульного тестирования упрощают проверку корректности работы программы [3].

Преимущества PyCharm:

- удобный интерфейс;
- богатые инструменты для анализа кода;
- поддержка Git и настройка среды под проект.

Недостатки PyCharm:

- высокие системные требования;
- платную профессиональную версию с расширенным функционалом [4].

Таким образом, для разработки игры «Палочки» язык Python подходит благодаря простоте синтаксиса, возможности быстрого прототипирования и наличию библиотек для графического интерфейса и реализации логики игры. Среда разработки PyCharm делает процесс создания программы более удобным, обеспечивая интеллектуальное автодополнение, отладку, управление проектом и интеграцию с системами контроля версий, что ускоряет разработку и облегчает сопровождение кода.

Вместе Python и PyCharm создают комфортную и эффективную платформу для реализации как учебного, так и полноценного игрового проекта. Скриншоты работы в выбранной среде приведены на рисунке 2.2

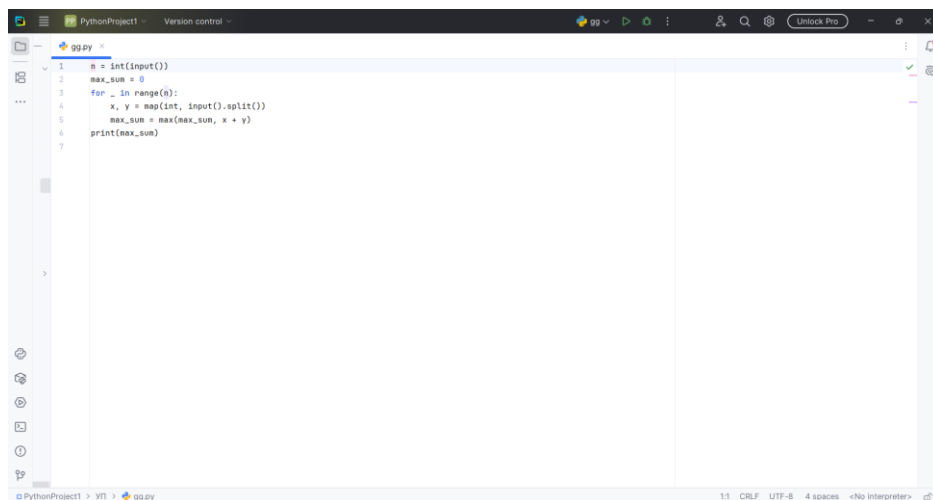


Рисунок 2.2 – Интерфейс PyCharm

Для системы контроля версий в соответствии с заданием использована система GitHub.

Ссылка на аккаунт: <https://github.com/sttaassp>

Ссылка на репозиторий для контроля: <https://github.com/sttaassp/UP.05>

Внешний вид аккаунта и репозитория в системе GitHub представлены на рисунках 2.3-2.4.

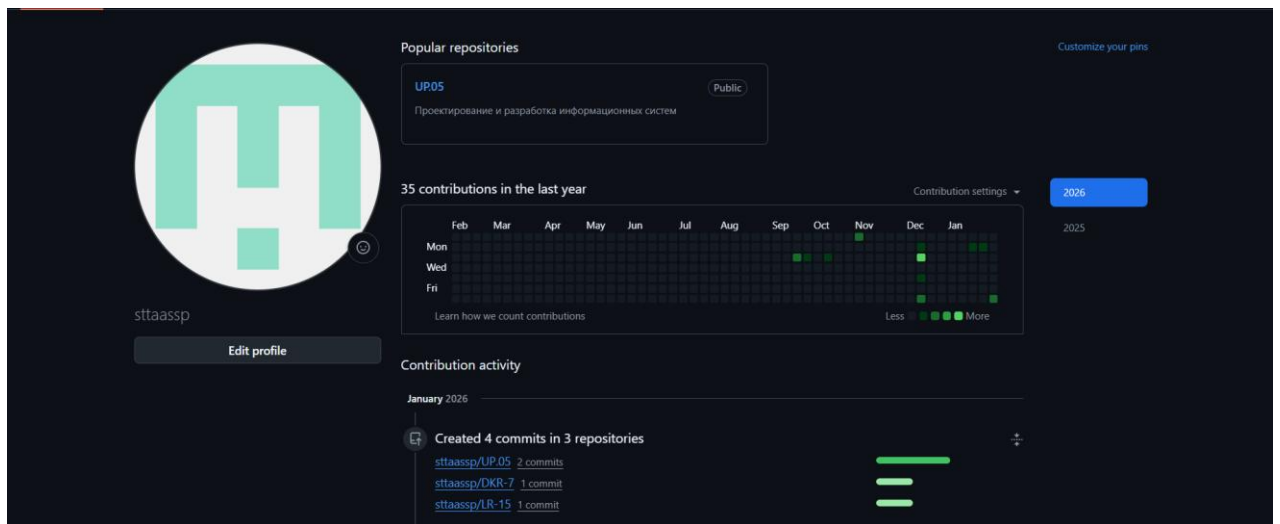


Рисунок 2.3 – Аккаунт в системе GitHub

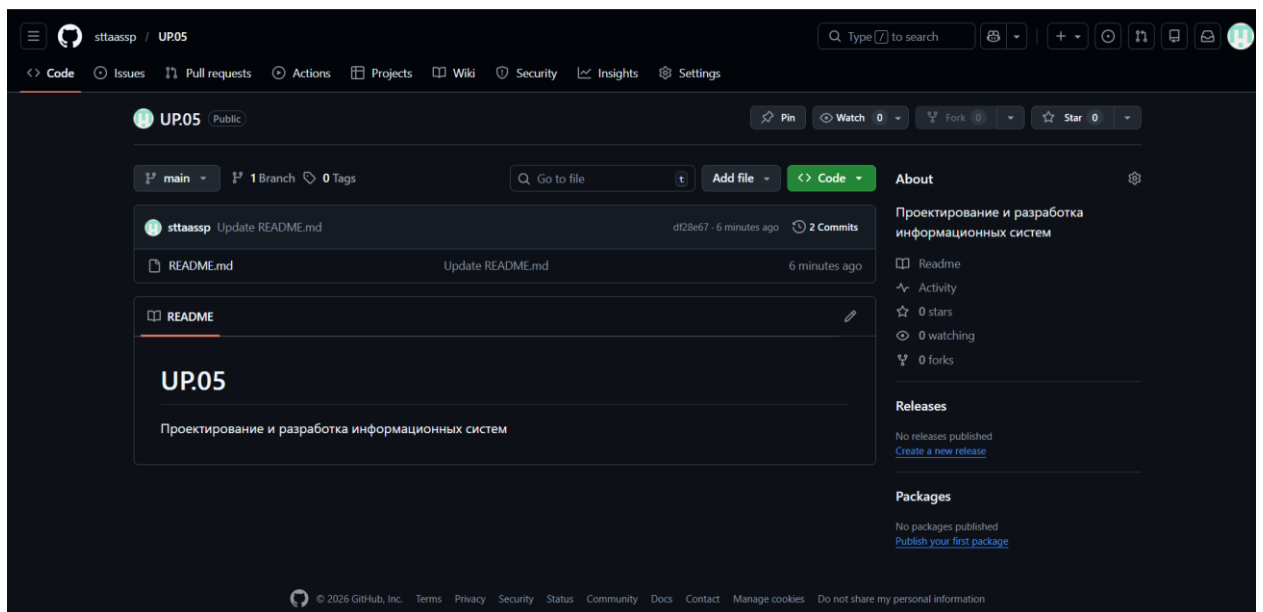


Рисунок 2.4 – Внешний вид репозитория в системе GitHub

4 ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Анализ предметной области

Игра «Палочки» относится к классу логических настольных (бумажных) игр стратегического характера, в которых результат полностью определяется действиями игроков. Данная игра широко используется как в развлекательной, так и в образовательной среде благодаря простоте правил и выраженному развивающему потенциалу. В основе игры лежит поочерёдное соединение точек линиями с целью формирования замкнутых квадратов на игровом поле [5].

Игровое поле представляет собой прямоугольную сетку точек, представленную на рисунке 4.1. Игроки по очереди проводят горизонтальные или вертикальные отрезки между соседними точками. Когда в результате хода замыкается квадрат, игрок получает очко и право на дополнительный ход. Победителем становится участник, набравший наибольшее количество квадратов к моменту завершения заполнения поля. Данная схема делает игру формально простой, но стратегически насыщенной [5].

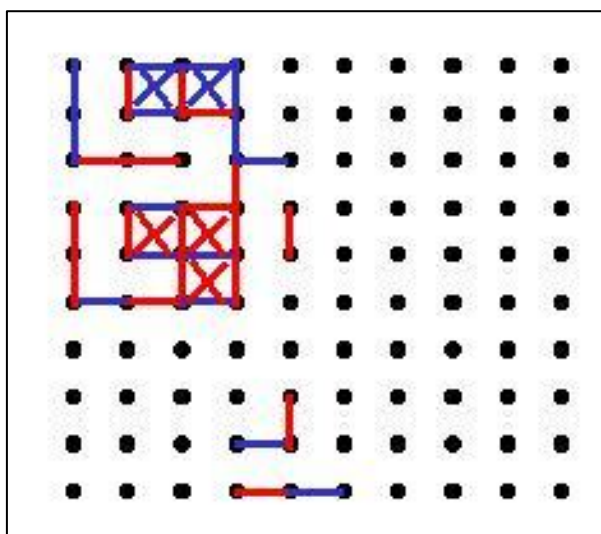


Рисунок 4.1 – Пример игрового поля

С точки зрения жанровой классификации игра относится к абстрактным стратегиям, поскольку в ней отсутствуют элементы случайности, а игровая ситуация полностью определяется предыдущими ходами игроков. Подобные игры характеризуются тем, что исход партии зависит исключительно от логического мышления, способности прогнозировать действия соперника и планировать собственные ходы [6].

Особенностью игры является наличие механизма цепочек: при формировании определённых конфигураций квадратов игрок может получить серию последовательных ходов. Таким образом, на поздних этапах партии ключевым становится не отдельный ход, а долгосрочное стратегическое планирование, связанное с контролем за формированием цепей квадратов. Это свойство придаёт игре глубину и делает её интересной не только для детей, но и для взрослых игроков [6].

Игра широко применяется в педагогической практике как средство развития математического и логического мышления. В образовательных материалах отмечается, что использование сеточных игр способствует формированию у обучающихся навыков анализа, прогнозирования и пространственного мышления. В частности, в методических разработках по обучению математике игра рассматривается как эффективный инструмент для знакомства с понятиями геометрических фигур, границ, замкнутых контуров и последовательностей действий [7].

Благодаря минимальным требованиям к инвентарю (достаточно листа бумаги и ручки) игра получила широкое распространение в школьной и внеклассной практике. Она используется как элемент развивающих занятий, математических кружков, а также как средство тренировки внимания и логического мышления в неформальной обстановке [5].

Для обобщённой оценки распространённости и жизненного цикла цифровых игр «Палочки» были проанализированы открытые данные магазинов мобильных приложений, игровых порталов и пользовательских сообществ. Поскольку разработчики не всегда публикуют точные

статистические показатели, в анализе используются усреднённые и оценочные данные.

Игра «Палочки» является одной из наиболее известных классических логических игр и существует в цифровом формате на протяжении нескольких десятилетий. Первые компьютерные реализации появились ещё в 1970–1980-х годах, а мобильные версии активно распространяются с начала 2010-х годов. Большинство современных мобильных приложений находятся в свободном доступе от 8 до 15 лет. Учебные, алгоритмические и исследовательские реализации существуют значительно дольше – более 30 лет – в рамках исследований теории игр, алгоритмов принятия решений и искусственного интеллекта. Средний срок «жизни» цифрового аналога можно оценить в 10–15 лет.

Мобильные версии игры, размещённые в Google Play и App Store, как правило, имеют от 50 000 до 500 000 загрузок. Наиболее популярные приложения превышают 1 миллион установок, что заметно выше показателей многих других логических настольных игр. Тем не менее, несмотря на широкую узнаваемость, игра остаётся преимущественно нишевой и не относится к массовым коммерческим продуктам.

Обновления для подобных проектов выходят нерегулярно. В среднем активная поддержка мобильных приложений продолжается 3–5 лет, после чего игра остаётся доступной, но развивается минимально и получает лишь технические исправления. Исключение составляют образовательные и исследовательские реализации, которые обновляются в рамках учебных курсов, научных экспериментов и соревнований по программированию.

Цифровые версии игры «Палочки» ориентированы преимущественно на:

- школьников и студентов;
- пользователей, интересующихся логическими, комбинаторными и математическими играми;

- разработчиков и исследователей в области алгоритмов, теории игр и искусственного интеллекта.

Массовая игровая аудитория проявляет умеренный интерес к подобным проектам, однако благодаря простоте правил, высокой стратегической глубине и образовательной ценности игра сохраняет стабильную популярность в цифровой среде и продолжает использоваться в учебных и исследовательских целях.

4.1.1 Анализ проблемы

Игра «Палочки» простая на правилах, но глубокая по стратегии двух пользовательская абстрактная игра, цель которой заключается в проведении рёбер между соседними точками на сетке так, чтобы замыкать квадраты и получать за это очки. Правила восходят к XIX веку и были формализованы как комбинаторная игра без случайности, где результат определяется исключительно выбором ходов игроков.

Основная сложность заключается в экспоненциальном росте числа возможных состояний игры. Для игрового поля размером $m \times n$ количество рёбер равно $p = m \cdot (n+1) + n \cdot (m+1)$, а число возможных состояний составляет 2^p , так как каждое ребро может быть проведено, либо отсутствовать. Кроме того, одно и то же состояние может быть достигнуто различными последовательностями ходов, что резко увеличивает количество перебираемых позиций при наивном поиске [8].

Дополнительную сложность представляет специфика выигрыша в данной игре. В отличие от большинства беспристрастных игр, где выигрывает игрок, сделавший последний ход, в «Точках и квадратах» результат определяется количеством захваченных квадратов. Это делает неприменимым классический аппарат теории беспристрастных игр, в частности теорему Шпрага – Гранди, широко используемую для анализа нормальных игр [8].

Кроме того, весь класс логических настольных игр, к которому относится «Палочки», традиционно рассматривается как средство развития логического мышления и стратегического анализа [9].

Таким образом, основная проблема при анализе заключается в следующем:

- экспоненциальный рост пространства состояний;
- нестандартная структура выигрыша;
- необходимость учитывать стратегические жертвы и цепочки;
- развитие познавательных навыков у игроков.

4.1.2 Обзор аналогов

Первым рассматриваемым аналогом является веб-игра «Dots and Boxes». Данный аналог представляет собой браузерную реализацию классической настольной игры, разработанную студией ConteZero. Игра распространяется бесплатно и доступна через веб-браузер без необходимости установки дополнительного программного обеспечения. Основная цель игры – поочерёдно соединять точки линиями, замыкая квадраты и получая за это очки.

Преимущества «Dots and Boxes»:

- доступность через любой современный веб-браузер без установки;
- простой и интуитивно понятный интерфейс;
- классическая реализация правил игры без искажений;
- возможность быстрой игры против компьютера.

Недостатки «Dots and Boxes»:

- ограниченные настройки игрового поля и сложности;
- отсутствие расширенных режимов игры;
- отсутствие обучающего режима и анализа ходов.

Пример работы с игрой «Dots and Boxes» от студии ConteZero представлен на рисунке 4.2

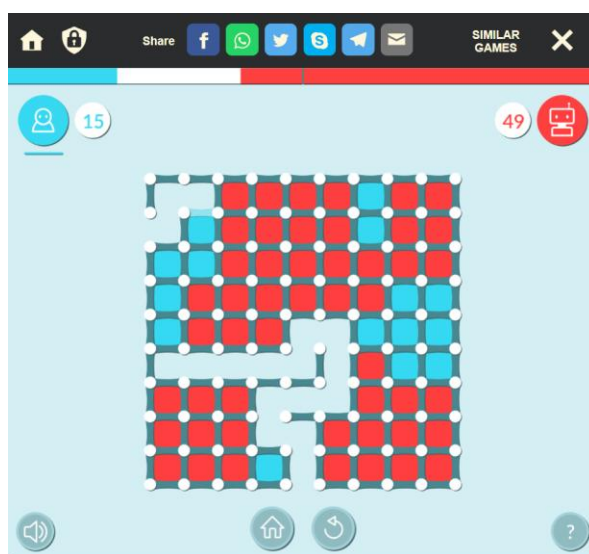


Рисунок 4.2 – Игровое поле «Dots and Boxes»

Следующим рассматриваемым аналогом станет мобильное приложение «Dots & Boxes – Classic Board Games», разработанное компанией OutOfTheBit Ltd. Данное приложение разработано компанией OutOfTheBit Ltd и распространяется через магазины Google Play и App Store. Программа представляет собой мобильную версию классической логической игры с поддержкой игры против искусственного интеллекта и локального мультиплеера.

Преимущества «Dots & Boxes – Classic Board Games»:

- поддержка мобильных устройств на базе android и ios;
- возможность игры против компьютера и против другого игрока на одном устройстве;
- наличие нескольких уровней сложности;
- удобный сенсорный интерфейс.

Недостатки «Dots & Boxes – Classic Board Games»:

- наличие рекламы в бесплатной версии;
- ограниченные возможности настройки поля;
- отсутствие сетевого мультиплеера.

Пример работы с мобильной игрой «Dots & Boxes – Classic Board Games» представлен на рисунке 4.3.

Третий рассматриваемый аналог – мобильная игра Dots Master, разработанная независимой студией и доступная на платформах Android и Windows. Игра представляет собой цифровую реализацию классической логической игры и ориентирована на широкую аудиторию пользователей.

Приложение поддерживает игру против компьютера, локальный многопользовательский режим, а также планируемый онлайн-режим. Пользовательский интерфейс выполнен в минималистичном стиле, что облегчает восприятие игрового поля и делает процесс игры интуитивно понятным. Игра предлагает разные уровни сложности: Новичок, Любитель и Профессионал. В игре реализована система подсчёта очков и автоматическое определение победителя.

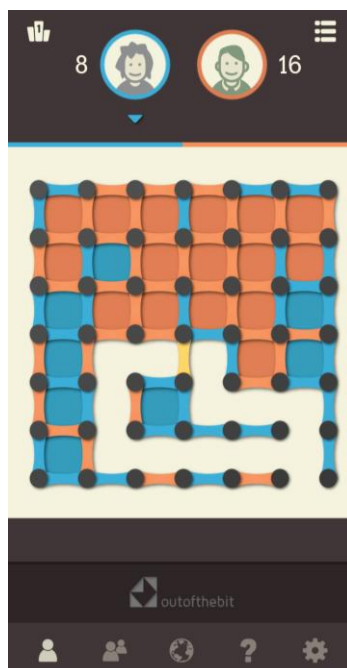


Рисунок 4.3 – Игровое поле приложения «Dots & Boxes – Classic Board Games»

Достоинства «Dots Master»:

- простота интерфейса и удобство управления;
- наличие режима игры против компьютера;
- поддержка локального многопользовательского режима;
- наличие разных уровней игры;

Недостатки «Dots Master»:

- ограниченные настройки сложности искусственного интеллекта;
- отсутствие расширенных обучающих режимов;
- недостаточно развитый онлайн-функционал;
- отсутствие аналитических инструментов для анализа стратегии игры.

Пример работы с мобильной игрой «Dots Master» представлен на рисунке 4.4.

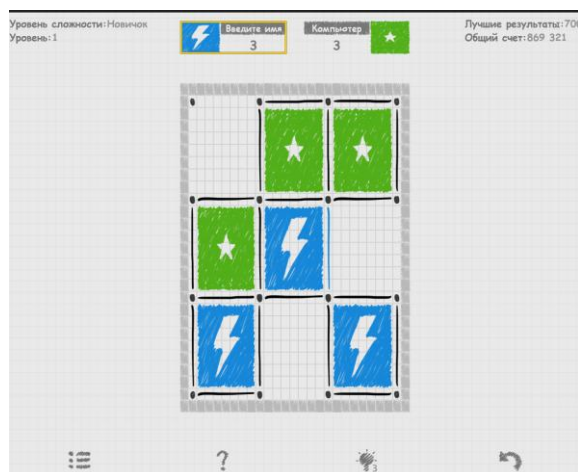


Рисунок 4.4 – Интерфейс мобильной версии игры «Dots Master»

Четвёртым рассматриваемым аналогом является браузерная игра «Drawing Squares». Данная игра является веб-реализацией, близкой по механике к «Dots Master», и ориентирована на развитие логического мышления. Игра доступна бесплатно и позволяет играть против компьютера или другого игрока.

Преимущества «Drawing Squares»:

- бесплатный доступ без установки;
- возможность выбора противника (человек или компьютер);
- простая графика и понятный интерфейс;
- быстрая загрузка и стабильная работа.

Недостатки «Drawing Squares»:

- ограниченные режимы игры;
- отсутствие сетевого режима;
- минимальные настройки сложности;
- отсутствие обучающих элементов.

Пример работы с браузерной игрой «Drawing Squares» представлен на рисунке 4.5.

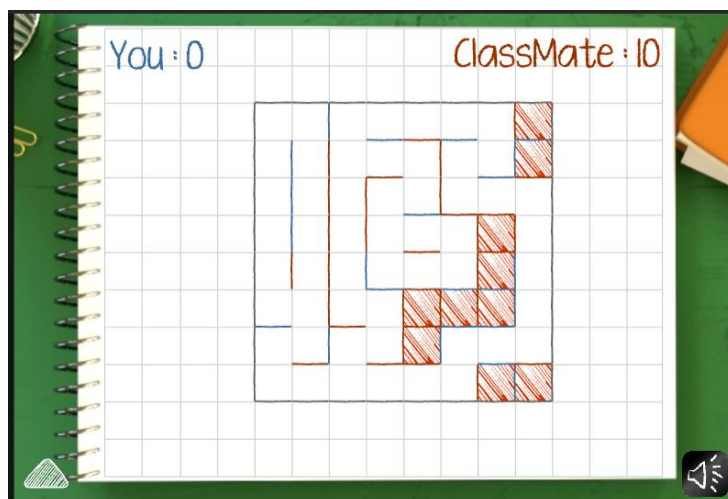


Рисунок 4.5 – Игровое поле «Drawing Squares»

Пятый рассматриваемый аналог – это веб-игра «Dots and Boxes 2», которая является онлайн-версия классической логической игры «Dots and Boxes».

Dots and Boxes 2 поддерживает правила оригинальной настольной игры и переносит её в удобный браузерный интерфейс – это позволяет играть как на ПК, так и на мобильных устройствах. Игра предлагает разные уровни сложности против компьютерного оппонента: Normal, Hard и Expert, что делает её подходящей как для новичков, так и для более опытных игроков.

Достоинства «Dots and Boxes 2»:

- не требует загрузки или установки, запускается в браузере;
- возможность играть с компьютером на разных уровнях сложности или с другом;
- настраиваемый размер игровой сетки;
- подходит как для мыши, так и для касаний сенсорного экрана.

Недостатки «Dots and Boxes 2»:

- отсутствие функции сетевой онлайн-игры с другими игроками через интернет;
- минимальные графические эффекты
- нет подсказок или обучающего режима для стратегий;

Пример работы с игрой «Dots and Boxes 2» представлен на рисунке 4.6.

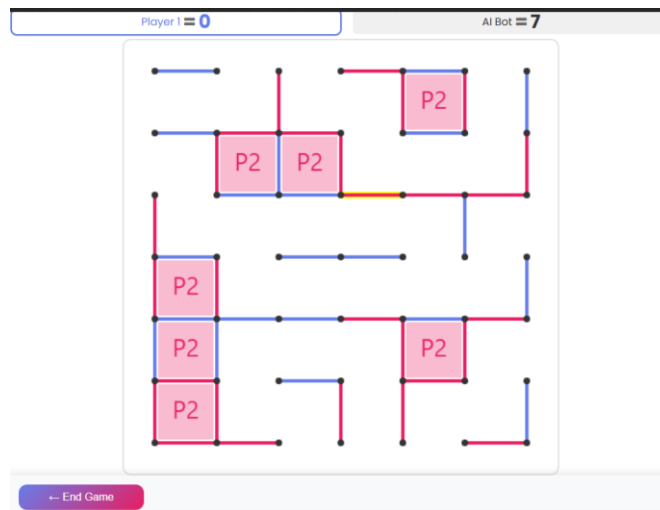


Рисунок 4.6 – Игровое поле «Dots and Boxes 2»

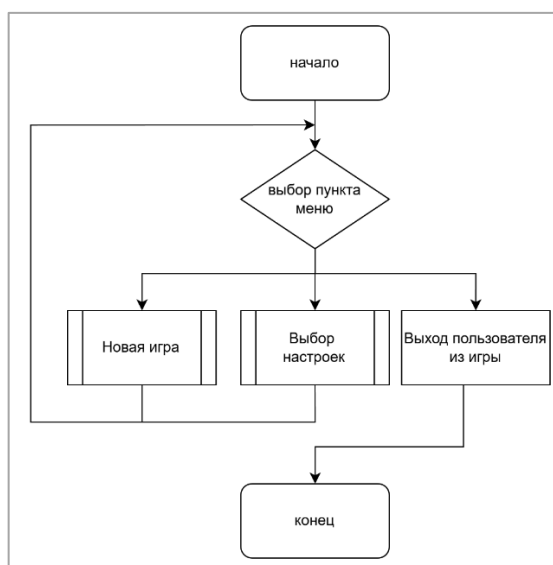
Таким образом, игра «Палочки», обладая простыми правилами, представляет собой стратегически сложную абстрактную игру, в которой важны планирование, прогнозирование ходов соперника и управление цепочками квадратов. Экспоненциальный рост числа возможных состояний существенно усложняет анализ игры и разработку эффективного компьютерного соперника.

Анализ существующих цифровых аналогов показал, что большинство из них ориентированы на базовую реализацию механики и удобство использования, однако имеют ограниченный функционал. В частности, в них отсутствуют развитые обучающие режимы, средства анализа ходов и поддержки стратегии, что подтверждает актуальность разработки собственного программного продукта, в котором пользователь сможет ознакомиться с правилами игры, настроить внешний вид приложения, выбирать режим игры и размеры игрового поля, а также регулировать уровень сложности компьютерного соперника.

4.2 Разработка структуры приложения и алгоритмов функционирования

4.2.1 Разработка алгоритмов функционирования

После запуска приложения происходит инициализация интерфейса: загружается главное окно со всеми элементами управления и кнопками навигации. Пользователю отображается главный экран с возможностью начать новую игру, перейти к просмотру правил или выйти из приложения. Алгоритм работы главного окна представлен на рисунке 4.2.1.



4.2.1 – Алгоритм работы главного окна игры «Палочки»

При переходе в режим выбора настроек пользователю предоставляется возможность изменить параметры приложения, включая смену языка интерфейса и цветового оформления, а также просмотр правил игры. После внесения необходимых изменений пользователь подтверждает выбор, после чего происходит возврат к предыдущему экрану или в главное меню. Алгоритм режима представлен на рисунке 4.2.2.



Рисунок 4.2.2 – Алгоритм режима «Выбор настроек»

При выборе пункта «Новая игра» пользователь переходит к экрану настройки параметров игры, где может выбрать режим (игра с ботом или игра с другим игроком). Алгоритм пункта «Новая игра» представлен на рисунке 4.2.3.

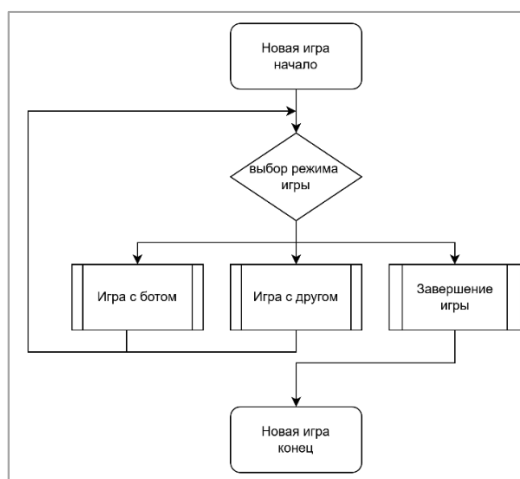


Рисунок 4.2.3. – Алгоритм пункта «Новая игра»

При выборе режима «Игра с ботом» пользователь задаёт уровень сложности, далее выбирает размер игрового поля, после чего определяется очередность ходов и начинается игровой процесс. Игрок и бот поочерёдно

выполняют ходы с проверкой условия завершения игры после каждого хода. При достижении определенных условий игра завершается. Алгоритм данного режима представлен на рисунке 4.2.4.

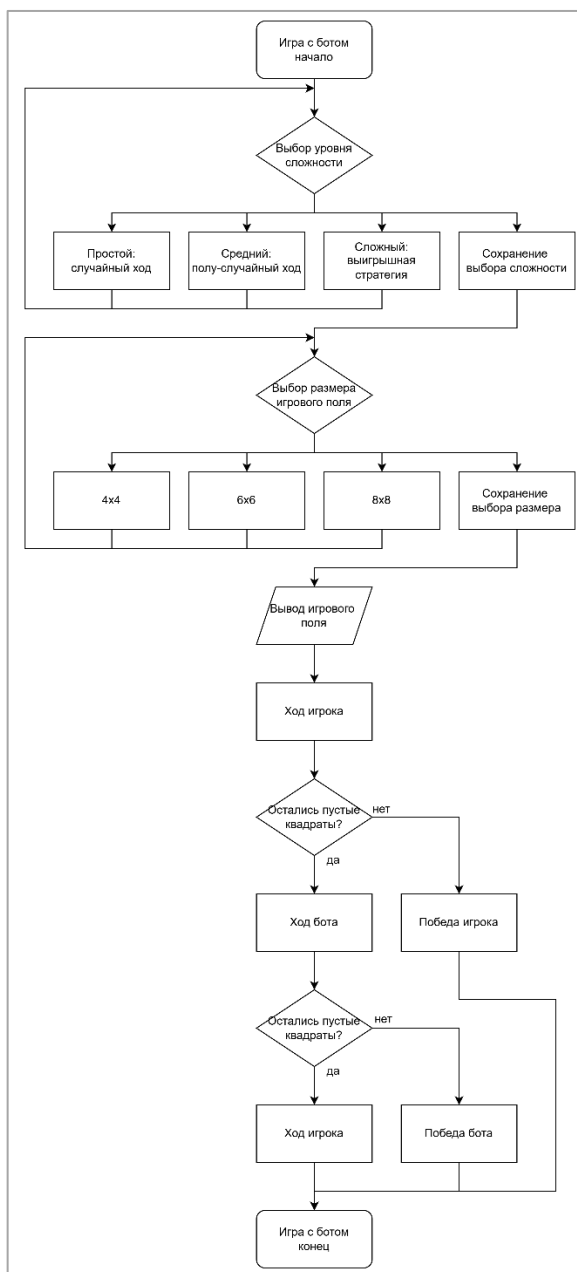


Рисунок 4.2.4 – Алгоритм режима «Игра с ботом»

При выборе режима «Игра с другом» пользователи сразу выбирают размеры игрового поля, далее определяется очередность ходов между двумя игроками, после чего начинается игровой процесс. Игроки поочерёдно выполняют ходы в соответствии с правилами игры «Палочки», при этом после

каждого хода осуществляется проверка условия завершения игры. Алгоритм режима «Игра с другом» представлен на рисунке 4.2.5.

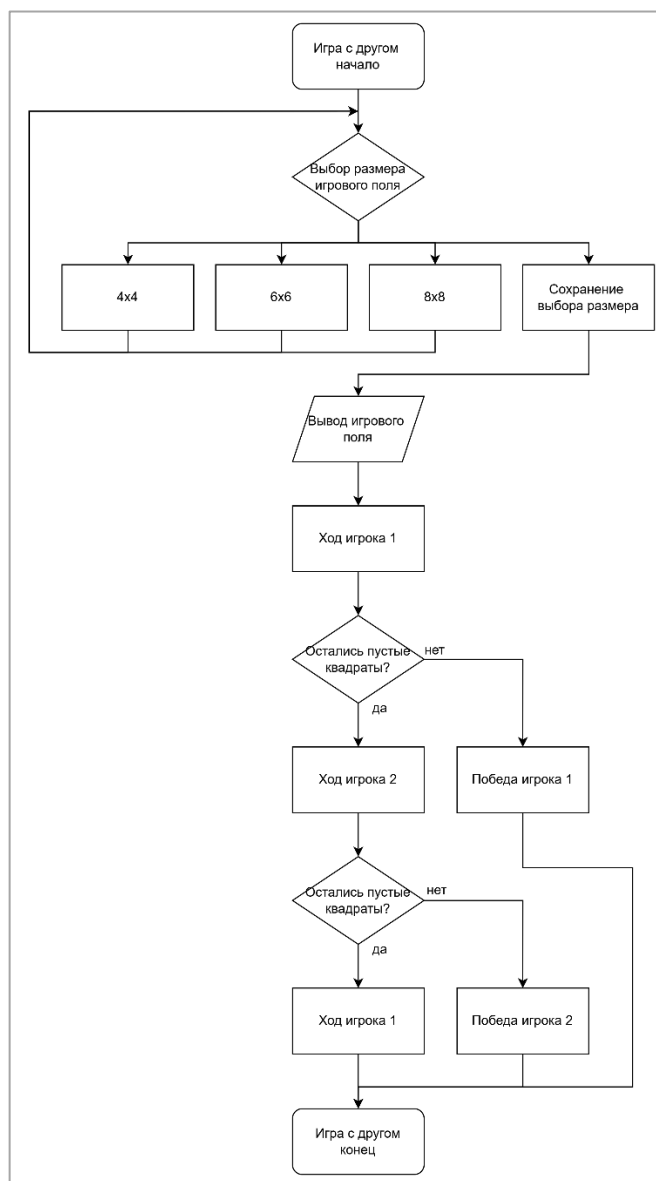


Рисунок 4.2.5 – Алгоритм режима «Игра с другом»

После окончания игровой партии выполняется вывод результатов игры. Пользователю предлагается выбрать дальнейшее действие: начать новую игру либо вернуться в главное меню. В зависимости от выбранного варианта осуществляется переход к соответствующему экрану, после чего режим завершения игры, представленный на рисунке 4.2.6, считается выполненным.



Рисунок 4.2.6 – Алгоритм режима «Завершение игры»

4.2.3 Проектирование интерфейса

Приводится прототип интерфейса, нарисованный в Figma. Прототип представлен на рисунке 4.3.1.

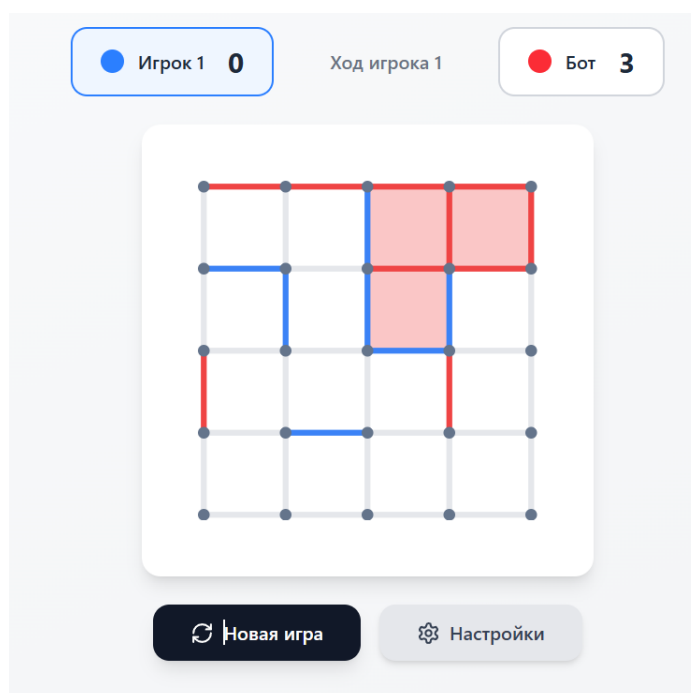


Рисунок 4.2.7 – Прототип интерфейса игры «Палочки»

В данном разделе рассмотрены основные алгоритмы работы игры «Палочки», включая навигацию по меню, выбор режимов игры и логику игрового процесса при игре с ботом и с другим пользователем, а также механизмы определения завершения игры и победителя.

Также представлен прототип пользовательского интерфейса, включающий визуализацию игрового экрана. Разработанные схемы алгоритма и прототип наглядно отражают логику работы системы и взаимодействие пользователя с приложением.

4.3 Разработка алгоритмов работы программы и программного обеспечения

В данном разделе будут произведены разработки алгоритмов функционирования игры «Палочки», разработан интерфейс программы, реализованы основные функции программы.

4.3.1 Разработка программного обеспечения

Программное обеспечение разработано в среде PyCharm на языке Python с использованием графической библиотеки tkinter. Архитектура приложения состоит из системы классов, разделяющих графический интерфейс, навигацию и игровую логику.

Основным компонентом системы является модуль `main.py`, в котором определен класс `SticksGame`. Данный класс выступает в роли центрального контроллера приложения: он инициализирует главное окно, хранит глобальные состояния (текущий язык интерфейса, выбранное цветовое оформление, настройки сложности и режима игры) и управляет переключением между экранными формами. Структура класса представлена на рисунке 4.3.1.

Функция `__init__(self)` – конструктор класса. Инициализирует полноэкранное окно приложения «Палочки» и создает базовый контейнер для всех экранных форм. Устанавливает глобальные переменные состояния: `current_lang = "Russian"`, `bg_color` (базовый цвет фона из конфигурации), а также инициализирует переменные `game_mode`, `game_size` и `game_diff` для хранения параметров текущей игровой сессии. В цикле создает экземпляры всех экранов приложения, сохраняя их в словарь `self.frames`, после чего вызывает метод `show_frame` для отображения главного меню.

```

import tkinter as tk

from config import BG_COLOR, LANGUAGES
from main_menu import MainMenu
from settings_menu import SettingsMenu
from exit_confirmation import ExitConfirmation
from mode_menu import ModeMenu
from difficulty_menu import DifficultyMenu
from size_menu import SizeMenu
from game_screen import GameScreen
from game_result import GameResult

class SticksGame(tk.Tk):
    """usage"""
    def __init__(self):
        ...

    def show_frame(self, page_name):
        ...

    def change_language(self, lang_code):
        ...

    def change_theme(self, color_hex):
        ...

    def refresh_all_frames(self):
        ...

if __name__ == "__main__":
    app = SticksGame()
    app.mainloop()

```

Рисунок 4.3.1 – Класс SticksGame

Функция `show_frame(self, page_name)` обеспечивает переключение между экранами приложения. Извлекает нужный объект из словаря `self.frames`, обновляет его содержимое через метод `refresh` (актуализация языка и темы) и выводит на передний план. При переходе к игровому полю дополнительно иницирует запуск игровой логики.

Функция `change_language(self, lang_code)` отвечает за смену локализации. Обновляет глобальную переменную `current_lang` выбранным значением и иницирует перерисовку всех текстовых элементов во всех экранных формах через метод `refresh_all_frames`.

Функция `change_theme(self, color_hex)` управляет визуальным оформлением. Устанавливает новый цвет фона для главного контейнера и вызывает каскадное обновление цветовой палитры для всех активных элементов в соответствии с выбранной темой.

Функция `refresh_all_frames(self)` выполняет служебную задачу по синхронизации интерфейса. В цикле обходит все зарегистрированные объекты

в словаре `self.frames` и вызывает их внутренние методы `refresh`, обеспечивая мгновенное применение настроек без перезагрузки приложения.

Класс `MainMenu` является стартовой экранной формой приложения. Он отвечает за визуализацию главного меню, навигацию между основными разделами игры (настройки, выбор режима) и инициализацию процедуры выхода. Структура класса представлена на рисунке 4.3.2.

```
import tkinter as tk
from config import *

class MainMenu(tk.Frame): 2 usages
    def __init__(self, parent, controller):...

    def refresh(self):...
```

Рисунок 4.3.2 – Класс `MainMenu`

Функция `__init__(self, parent, controller)` – конструктор класса. Инициализирует графическую панель с заданным цветом фона. Размещает кнопку «Настройки» в углу экрана, центральный заголовок игры, блок кнопок «Новая игра» и «Выход». Настраивает логику переключения кадров через объект `controller`.

Функция `refresh(self)` – метод динамического обновления интерфейса. Обновляет отображаемый текст из словаря `LANGUAGES` для заголовка и названий кнопок с выбранным языком локализации. Обновляет цветовую схему виджетов из словаря `THEME`, автоматически устанавливая акцентный цвет для кнопки начала игры в зависимости от выбранной темы оформления.

Класс `ModeMenu` представляет собой экранную форму для выбора режима игры. Пользователь может выбрать два режима («Игра с ботом» или

«Игра с другом»), а также перемещаться по экранам с помощью кнопок «Назад» и «Далее». Структура класса представлена на рисунке 4.3.3.

```
import tkinter as tk
from config import *

class ModeMenu(tk.Frame): 2 usages
    def __init__(self, parent, controller):...

    def _create_mode_option(self, text, value):...

    def refresh(self):...

    def _on_next(self):...
```

Рисунок 4.3.3 – Класс ModeMenu

Функция `__init__(self, parent, controller)` – конструктор класса. Инициализирует графическую панель и настраивает систему хранения выбранных параметров. Формирует визуальную структуру окна, размещает заголовок, центральный контейнер, в котором создаются карточки доступных режимов («Игра с ботом» или «Игра с другом»), кнопки «Назад» и «Далее» для навигации по экранным формам.

Функция `_create_mode_option(self, text, value)` – вспомогательный метод для создания интерактивных элементов выбора. Настраивает обработчики событий клика, которые обеспечивают визуальную индикацию выбора (подсветку рамки) и разблокировку кнопки «Далее» для перехода к следующему окну.

Функция `refresh(self)` – метод динамического обновления интерфейса. Обновляет отображаемый текст из словаря `LANGUAGES` для заголовка и названий кнопок с выбранным языком локализации. Обновляет цветовую схему виджетов из словаря `THEME`, автоматически устанавливая акцентный цвет для кнопки начала игры в зависимости от выбранной темы оформления.

Функция `_on_next(self)` управляет логикой навигации. В зависимости от выбранного режима игры определяет следующий этап настройки: перенаправляет пользователя либо на экран выбора сложности бота, либо сразу к выбору размера игрового поля.

Класс `DifficultyMenu` представляет собой экранную форму для выбора уровня сложности компьютерного соперника. Она позволяет пользователю настроить уровень сложности логики оппонента при игре в режиме против бота. Структура класса представлена на рисунке 4.3.4.

```
import tkinter as tk
from config import *

class DifficultyMenu(tk.Frame): 2 usages
    def __init__(self, parent, controller):...

    def _create_difficulty_option(self, text, value):...

    def refresh(self):...

    def _on_next(self):...
```

Рисунок 4.3.4 – Класс `DifficultyMenu`

Функция `__init__(self, parent, controller)` – конструктор класса, инициализирующий графическую панель и задающий переменные состояния. В рамках функции создается объект `self.selected_diff` для хранения выбранной сложности и словари `self.option_frames`, `self.option_labels` для управления виджетами. Формируются кнопки «Назад» и «Далее» для навигации по экранным формам.

Функция `_create_mode_option(self, text, value)` – вспомогательный метод для создания интерактивных элементов выбора сложности бота («Простой», «Средний» и «Сложный»). Настраивает обработчики событий клика, которые обеспечивают визуальную индикацию выбора (подсветку рамки) и разблокировку кнопки «Далее» для перехода к следующему окну.

Функция `refresh(self)` – метод динамического обновления интерфейса. Обновляет отображаемый текст из словаря `LANGUAGES` для заголовка и названий кнопок с выбранным языком локализации. Обновляет цветовую схему виджетов из словаря `THEME`, автоматически устанавливая акцентный цвет для кнопки начала игры в зависимости от выбранной темы оформления.

Функция `_on_next(self)` – логический обработчик перехода, который фиксирует финальный выбор пользователя. Она передает установленное значение сложности в глобальные настройки контроллера через `self.controller.game_diff`.

Класс `SizeMenu` представляет собой экранную форму для настройки размеров игрового поля. Она позволяет пользователю выбрать один из трех доступных вариантов сетки перед запуском игрового процесса. Структура класса представлена на рисунке 4.3.5.

```
import tkinter as tk
from config import *

class SizeMenu(tk.Frame): 2 usages
    def __init__(self, parent, controller):...

    def _create_size_option(self, text, value):...

    def refresh(self):...

    def _on_back(self):...

    def _on_next(self):...
```

Рисунок 4.3.5 – Класс `SizeMenu`

Функция `__init__(self, parent, controller)` – конструктор класса, инициализирующий графическую панель и задающий переменные состояния. Инициализирует переменную `self.selected_size` для хранения выбора и словари `self.option_frames`, `self.option_labels` для управления виджетами. Размещает заголовок, кнопки «Назад» для возврата на предыдущие экранные формы и кнопка «Запустить игру» для начала игрового процесса.

Функция `_create_size_option(self, text, value)` – вспомогательный метод для создания интерактивных элементов выбора размера игрового поля («4x4», «6x6» и «8x8»). Настраивает обработчики событий клика, которые обеспечивают визуальную индикацию выбора (подсветку рамки) и разблокировку кнопки «Далее» для перехода к следующему окну.

Функция `refresh(self)` – метод динамического обновления интерфейса. Обновляет отображаемый текст из словаря `LANGUAGES` для заголовка и названий кнопок с выбранным языком локализации. Обновляет цветовую схему виджетов из словаря `THEME`, автоматически устанавливая акцентный цвет для кнопки начала игры в зависимости от выбранной темы оформления.

Функция `_on_back(self)` – логический обработчик, перенаправляющий пользователя на экран `DifficultyMenu` при игре с ботом или на `ModeMenu` в режиме против друга, обеспечивая корректный возврат в зависимости от выбранного сценария.

Функция `_on_next(self)` – финальный обработчик выбора. Сохраняет выбранный размер поля в глобальную переменную контроллера `self.controller.game_size` и инициирует переход к основной экранной форме игрового процесса `GameScreen`.

Класс `GameLogic` является расчетным модулем, отвечающим за математическую модель игры, хранение состояния игрового поля и алгоритмы поведения компьютерного соперника. Структура класса представлена на рисунке 4.3.6.

Функция `__init__(self, size)` – конструктор логики. Инициализирует параметры сетки – количество точек `self.n`, создает словарь `self.lines` для регистрации занятых линий и устанавливает начальный счет для обоих игроков в словаре `self.scores`.

Функция `get_all_possible_lines(self, offset, step)` является аналитическим методом, который сканирует всё игровое поле и создает список координат всех свободных на данный момент горизонтальных и вертикальных линий, учитывая текущие отступы и шаг сетки.

```

import random

class GameLogic: 2 usages
    def __init__(self, size):...

    def get_all_possible_lines(self, offset, step):...

    def check_squares_closure(self, line, player_id, offset, step):...

    def get_bot_move(self, difficulty, offset, step):...

    def _creates_opportunity(self, line, offset, step):...

```

Рисунок 4.3.6 – Класс GameLogic

Функция `check_squares_closure(self, line, player_id, offset, step)` – ключевой метод проверки правил. После каждого хода он определяет, образовали ли занятые линии квадрат. В случае успеха функция увеличивает счет игрока на единицу, фиксирует координаты захваченных ячеек и возвращает флаг необходимости дополнительного хода.

Функция `get_bot_move(self, difficulty, offset, step)` – управляет поведением компьютерного оппонента. В зависимости от выбранного уровня сложности она выбирает либо случайную свободную линию, либо ищет ход, приносящий очки, либо анализирует поле для предотвращения выгодных ходов со стороны игрока.

Функция `_creates_opportunity(self, line, offset, step)` – вспомогательный метод анализа игрового поля, предназначенный для реализации алгоритма высокой сложности. Она временно фиксирует линию в словаре `self.lines`, чтобы оценить, не приведет ли этот шаг к созданию ситуации, при которой у противника появится возможность закрыть квадрат своим следующим ходом.

Класс `GameScreen` – центральный компонент приложения, реализующий графический интерфейс игрового процесса, визуализацию хода игры и взаимодействие с пользователем через холст (Canvas). Структура класса представлена на рисунке 4.3.7.

```

import tkinter as tk
from config import *
from game_logic import GameLogic

class GameScreen(tk.Frame): 2 usages
    def __init__(self, parent, controller):...

    def refresh(self):...

    def start_game(self):...

    def _draw_grid(self, n):...

    def _on_canvas_click(self, event):...

    def _find_closest_line(self, x, y):...

    def _make_move(self, line):...

    def _bot_turn_logic(self):...

    def _update_ui(self):...

    def _show_final_results(self):...

```

Рисунок 4.3.7 – Класс GameScreen

Функция `__init__(self, parent, controller)` – конструктор класса. Формирует верхнюю панель счета (header) с именами игроков и индикатором хода, центральное игровое поле (canvas) и нижнюю панель управления (footer) с кнопками «Новая игра» и «Главное меню». Также инициализирует параметры отрисовки сетки (step, offset).

Функция `refresh(self)` – метод динамического обновления интерфейса. Обновляет отображаемый текст из словаря `LANGUAGES` для имен игроков, статусных сообщений и названий кнопок с выбранным языком локализации. Обновляет цветовую схему виджетов из словаря `THEME`, автоматически устанавливая акцентный цвет для кнопок «Новая игра» и «Главное меню».

Функция `start_game(self)` инициирует начало или перезапуск матча. Создает новый экземпляр `GameLogic`, сбрасывает счет и очередность ходов, рассчитывает размеры холста под выбранную сетку и создает базовые точки игрового поля методом `_draw_grid`.

Функция `_on_canvas_click(self, event)` представляет собой обработчик клика мыши по игровому полю. Определяет координаты ближайшей линии

через `_find_closest_line` и, если ход допустим, вызывает метод `_make_move`. Блокирует ввод игрока во время хода бота.

Функция `_make_move(self, line)` реализует механику совершения хода. Она регистрирует линию в логике, отрисовывает её на холсте цветом активного игрока, выполняет закраску захваченных квадратов (при их наличии) и переключает ход либо иницирует ход бота.

Функция `_update_ui(self)` обновляет информационные панели. Актуализирует числовые значения счета и визуально выделяет рамку того игрока (синим или красным цветом), чей ход является активным в данный момент.

Функция `_bot_turn_logic(self)` управляет задержкой и выполнением хода искусственного интеллекта. Запрашивает у логического модуля координаты линии в соответствии с уровнем сложности и применяет их к игровому полю.

Класс `GameResult` – экранная форма, предназначенная для отображения итогов матча, объявления победителя и предоставления навигации после завершения игры. Структура класса представлена на рисунке 4.3.8.

```
import tkinter as tk
from config import *

class GameResult(tk.Frame): 2 usages
    def __init__(self, parent, controller):...

    def refresh(self, s1=None, s2=None):...

    def _create_score_box(self, parent, score, label, color):...
```

Рисунок 4.3.8 – Класс `GameResult`

Функция `__init__(self, parent, controller)` – конструктор класса. Устанавливает связь с главным контроллером приложения и инициализирует базовые параметры графической панели.

Функция `refresh(self, s1=None, s2=None)` – основной метод построения интерфейса результатов. Она очищает экран, определяет победителя на основе сравнения очков `s1` и `s2`, и динамически формирует центральную карточку `card`. Метод обновляет все текстовые сообщения («Победил!», «Ничья!», «Игра завершена») на выбранный язык и перекрашивает элементы в зависимости от того, какой игрок одержал победу.

Функция `_create_score_box(self, parent, score, label, color)` – вспомогательный графический метод. Создает стилизованные информационные блоки для каждого игрока, отображая их финальный счет крупным шрифтом и подписывая имя соответствующим цветом – синим для Игрока 1 и красным для оппонента.

Класс `SettingsMenu` – экранная форма, позволяющая пользователю изменять язык и цветовое оформление интерфейса, а также ознакомиться с правилами игры. Структура класса представлена на рисунке 4.3.9.

A screenshot of a code editor showing the definition of the SettingsMenu class. The code is as follows:

```
import tkinter as tk
from config import *

class SettingsMenu(tk.Frame): 2 usages
    def __init__(self, parent, controller):...

    def refresh(self):...
```

Рисунок 4.3.9 – Класс `SettingsMenu`

Функция `__init__(self, parent, controller)` – конструктор класса. Устанавливает начальный цвет фона панели, соответствующий настройкам контроллера, и вызывает метод `refresh` для первичного построения интерфейса.

Функция `refresh(self)` – основной метод управления визуальным состоянием. Он полностью очищает текущую панель и заново отрисовывает

все элементы – кнопку возврата в меню, центральную карточку с заголовком, селекторы выбора языка и цветовых схем.

Класс `ExitConfirmation` – экранная форма модального типа, которая запрашивает у пользователя подтверждение перед закрытием программы. Структура класса представлена на рисунке 4.3.10.

Функция `__init__(self, parent, controller)` — конструктор класса, который создает визуальную карточку `card` в центре экрана. Она инициализирует текстовый вопрос `self.label_question` и две управляющие кнопки: `self.btn_cancel` для возврата в главное меню и `self.btn_confirm` для системного завершения работы приложения через `controller.quit`.

```
import tkinter as tk
from config import *

class ExitConfirmation(tk.Frame): 2 usages
    def __init__(self, parent, controller):...

    def refresh(self):...

    def _on_cancel(self):...
```

Рисунок 4.3.10 – Класс `ExitConfirmation`

Функция `refresh(self)` – метод динамического обновления интерфейса. Обновляет отображаемый текст из словаря `LANGUAGES` для вопроса и названий кнопок с выбранным языком локализации. Обновляет цветовую схему виджетов из словаря `THEME`, автоматически устанавливая акцентный цвет для кнопки начала игры в зависимости от выбранной темы оформления.

Функция `_on_cancel(self)` представляет собой вспомогательный метод навигации, который обеспечивает безопасный возврат пользователя на главный экран приложения при отказе от выхода.

Так же на рисунке 4.3.11 представлена диаграмма классов.

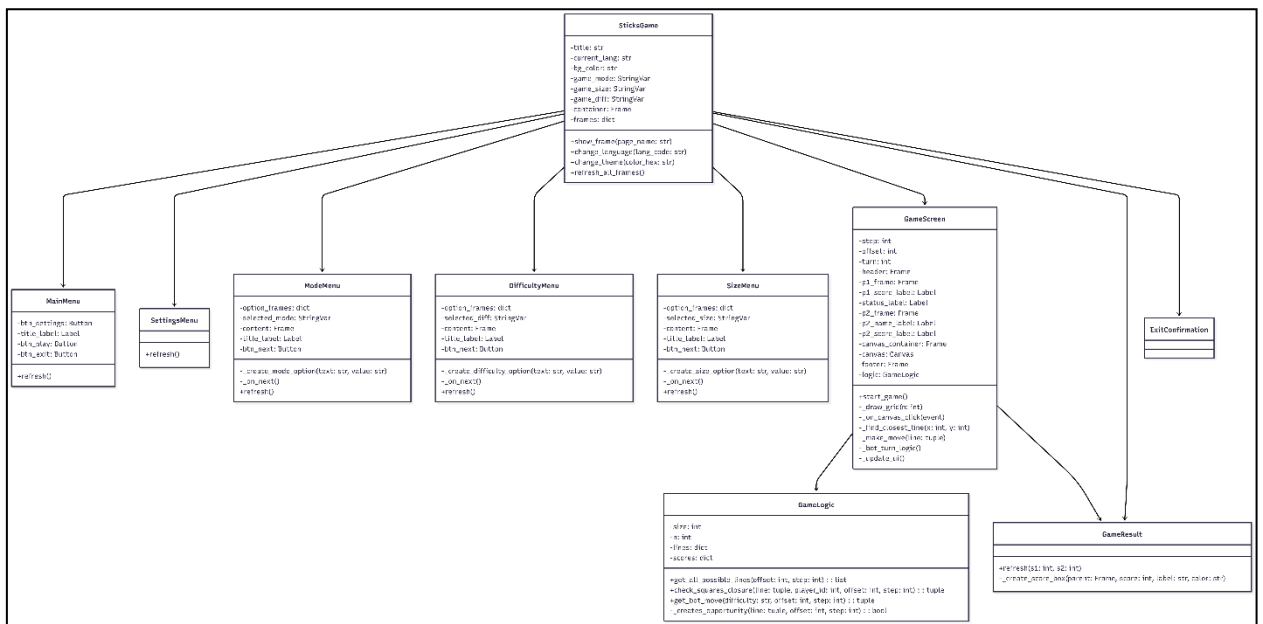


Рисунок 4.3.11 – Диаграмма классов

4.3.2 Разработка пользовательского интерфейса

Пользовательский интерфейс разработан с использованием библиотеки tkinter. Все экранные формы выполнены в едином современном стиле: используются настраиваемые цветовые темы (голубая, светло-серая, бирюзовая), минималистичные карточки (Frame) и контрастные элементы управления. Основные компоненты интерфейса: Button (кнопки навигации), Label (информационные метки и заголовки) и Canvas (интерактивное игровое поле).

Главное меню – стартовая форма приложения, созданная классом MainMenu. Содержит кнопки «Играть», «Настройки» и «Выход». Интерфейс реализован центрированным блоком кнопок на основном фоне. Внешний вид в русскоязычной версии представлен на рисунке 4.3.12.

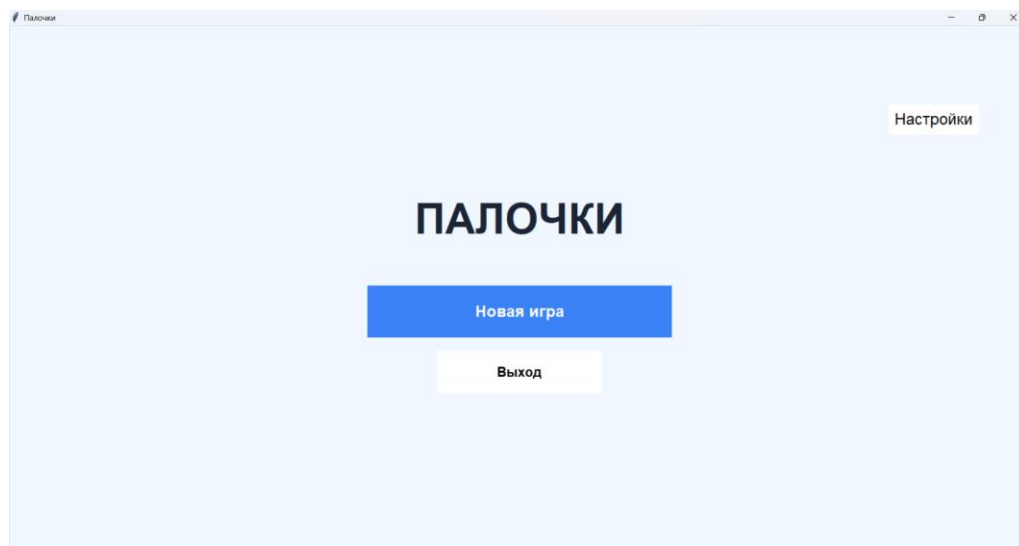


Рисунок 4.3.12 – Главное меню приложения

Экран настроек создается классом `SettingsMenu`. С помощью кнопок `Button`, осуществляется переключение между русским и английским языком. Выбор цветовой схемы реализован через динамическую генерацию кнопок в цикле, которые при нажатии вызывают функцию `change_theme()`. При изменении параметров метод `refresh()` мгновенно обновляет оформление карточки и все текстовые метки (`Label`), включая информационный блок «Об игре». Внешний вид представлен на рисунке 4.3.13.

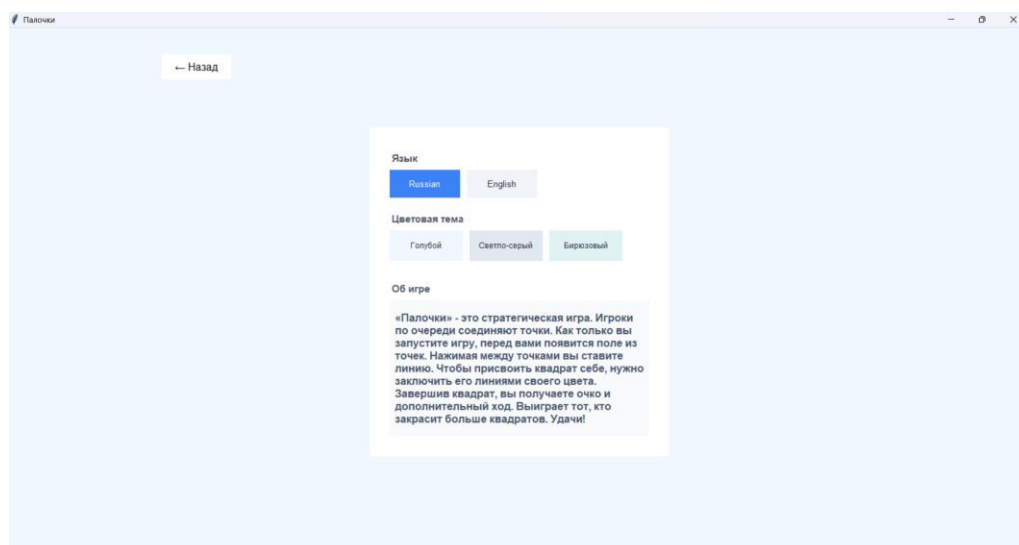


Рисунок 4.3.13 – Экран настроек

Экран выбора режима создается классом `ModeMenu`. С помощью кнопок `Button` пользователю предоставляется выбор между двумя сценариями игры «Игра с ботом» и «Игра с другом». При нажатии на кнопку «Игра с ботом» переменной `game_mode` присваивается соответствующее значение, и вызывается функция для перехода к настройкам сложности. При выборе режима «Игра с другом» приложение перенаправляет пользователя сразу к выбору размера поля. Внешний вид экрана выбора режима представлен на рисунке 4.3.14.

Экран выбора сложности создается классом `DifficultyMenu`. С помощью динамически создаваемых карточек `Frame` пользователю предлагается три уровня сложности: «Простой», «Средний» и «Сложный». Каждая карточка содержит иконку и текстовую метку `Label`. При клике на карточку срабатывает внутренняя функция `select`, которая подсвечивает рамку активным цветом и переводит кнопку «Далее» (`Button`) в активное состояние. Внешний вид экрана выбора сложности бота представлен на рисунке 4.3.15.

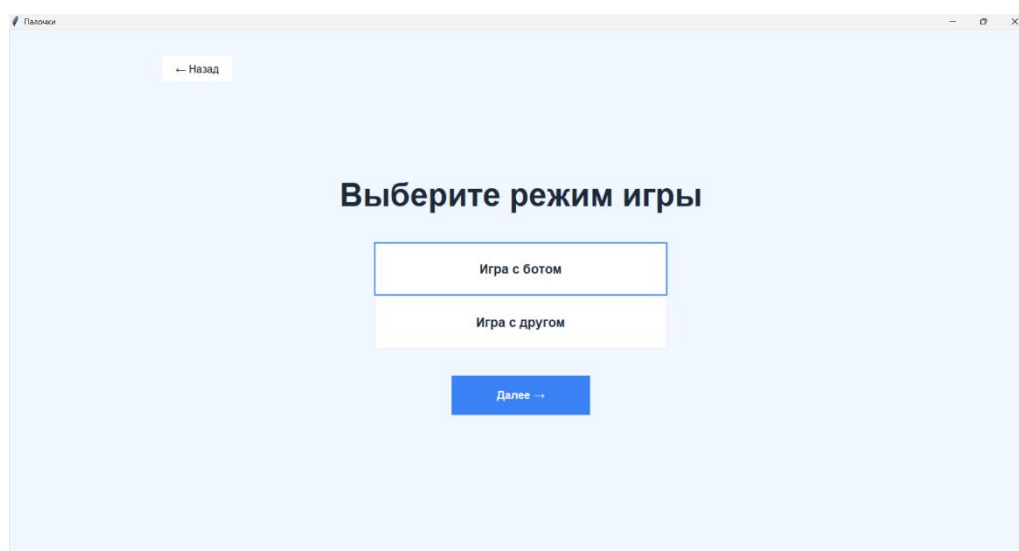


Рисунок 4.3.14 – Экран выбора режима игры

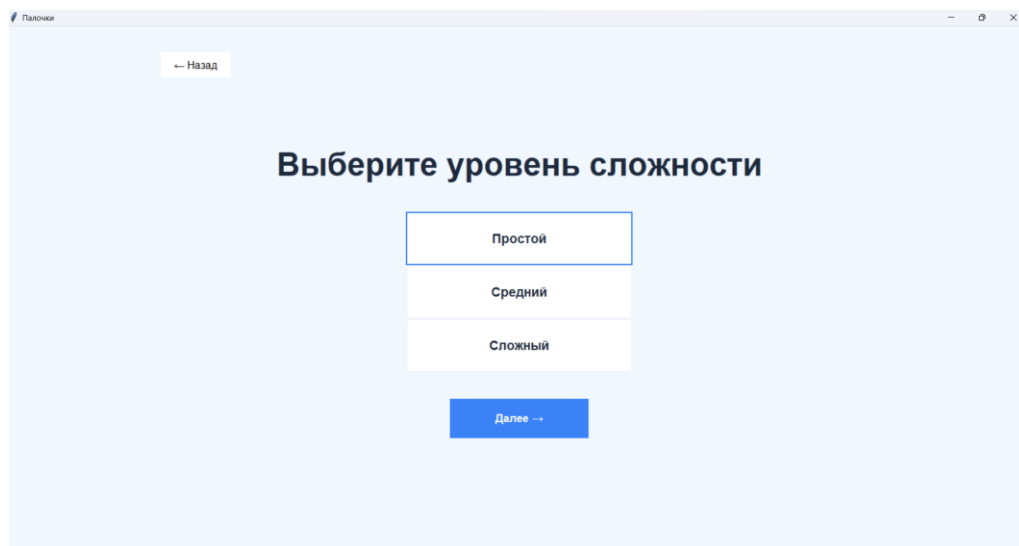


Рисунок 4.3.15 – Экран выбора сложности бота

Экран выбора размера игрового поля создается классом `SizeMenu`. С помощью динамически создаваемых карточек `Frame` пользователю предлагается три размера игрового поля : «4x4», «6x6» и «8x8». Функция `_create_size_option()` формирует графические элементы и привязывает к ним события выбора. Обработчик кнопки «Назад» (`_on_back`) реализует навигацию, возвращая пользователя либо к выбору сложности, либо к выбору режима игры. Внешний вид представлен на рисунке 4.3.16.

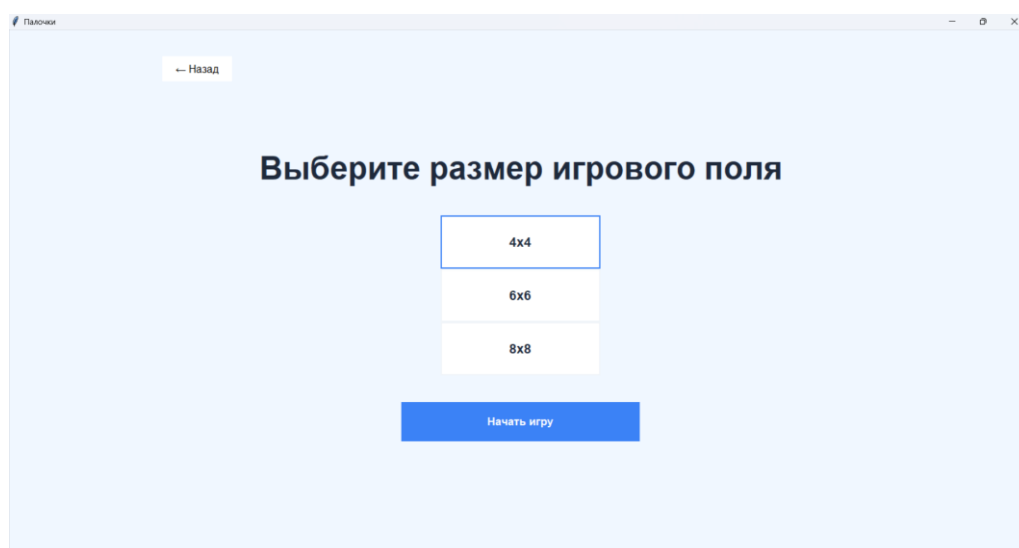


Рисунок 4.3.16 – Экран выбора размера игрового поля

Игровой экран создается классом `GameScreen`. Панель состояния, реализованная через `Frame` и `Label`, отображает текущий счет и имена игроков.

Центральным элементом является холст Canvas, на котором функция `start_game()` отрисовывает сетку точек. Метод `_on_canvas_click()` обрабатывает нажатия пользователя, а функция `_update_ui()` динамически переключает цветовую индикацию активного хода между синим (Игрок 1) и красным (Бот/Игрок 2) цветами. Внешний вид представлен на рисунке 4.3.17.

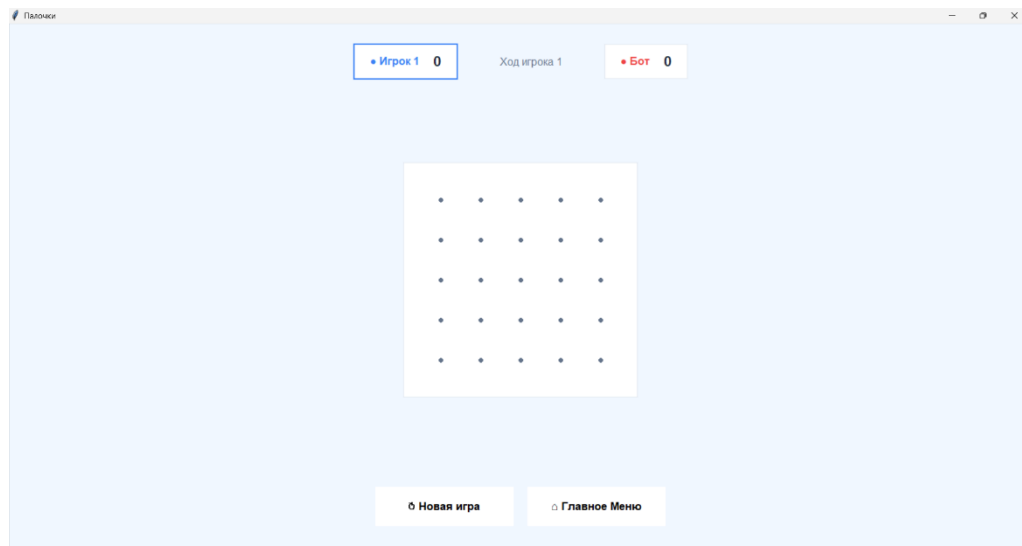


Рисунок 4.3.17 – Игровой экран

Экран результатов игры создается классом `GameResult`. Функция `refresh()` анализирует итоговый счет из `GameLogic` и формирует поздравительное сообщение. С помощью вспомогательного метода `_create_score_box()` создаются контрастные блоки с финальными очками каждого игрока. Кнопки навигации позволяют мгновенно начать новую партию или вернуться в меню. Внешний вид представлен на рисунке 4.3.18.

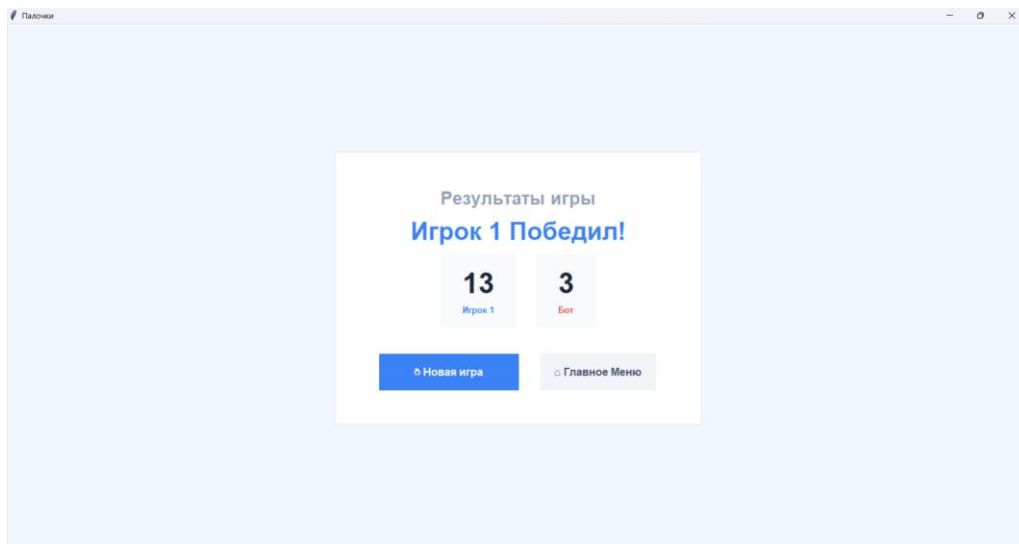


Рисунок 4.3.18 – Экран отображения результатов игры

Экран подтверждения выхода реализован классом `ExitConfirmation` в виде оверлейного окна. С помощью `Label` выводится вопрос о выходе, а кнопка `Button` с текстом «Подтвердить» вызывает системную функцию `quit()` для закрытия приложения. Метод `refresh()` гарантирует соответствие цветовой гаммы окна выбранной в настройках теме. Внешний вид представлен на рисунке 4.3.19.

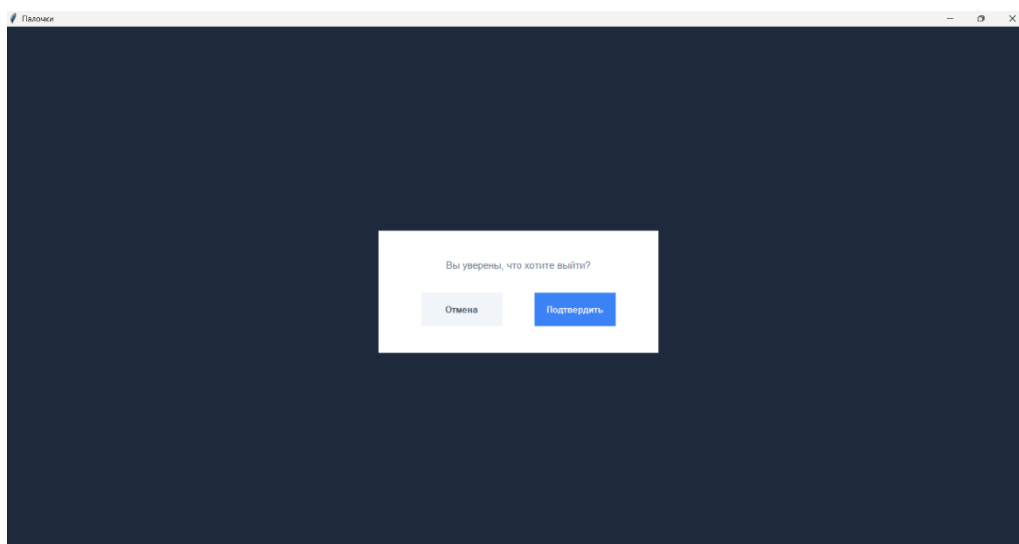


Рисунок 4.3.19 – Экран подтверждения выхода

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Какой язык программирования выбрать? Обзор Python // URL: <https://habr.com/ru/companies/kryptonite/articles/807345/> (дата обращения: 31.01.2026)
2. Основные недостатки языка Python // URL: <https://habr.com/ru/articles/504126/> (дата обращения: 31.01.2026)
3. Основы PyCharm // URL: <https://habr.com/ru/articles/720480/> (дата обращения: 31.01.2026)
4. 10 причин, почему мы перешли на PyCharm // URL: <https://habr.com/ru/articles/122018/> (дата обращения: 31.01.2026)
5. Точки и квадраты // URL: <https://smart-kids.su/igry/na-bumage/tochki-i-kvadratny> (дата обращения: 24.01.2026)
6. Dots and Boxes // URL: <https://mathforlove.com/lesson/dots-and-boxes-game> (дата обращения: 24.01.2026)
7. Настольная стратегическая игра «Точки и квадраты» // URL: <https://www.thegamersguides.com/dots-and-boxes-classic-strategy-board-games/> (дата обращения: 24.01.2026)
8. Barker & Korf, «Solving Dots-And-Boxes»
9. Логические игры – развиваем логику // URL: https://habr.com/ru/hubs/logic_games/articles (дата обращения: 24.01.2026)